

Deep Hedging

Riccardo Ravelli Andrea Rossini Alessio Scrazzolo

Guido Torrisi Mattia Zanin

September 5, 2026

Abstract

Hedging a portfolio of derivatives is one of the main objectives of quantitative finance. The classical solution computes model-based sensitivities, the *Greeks*, and continuously rebalances a position in the underlying to neutralize them. This method assumes a frictionless, complete market and is far from optimal once transaction costs, liquidity constraints and trading limits are taken into account. In this report we study *deep hedging*, a fully data-driven alternative in which a neural network learns the hedging strategy directly from data by minimizing a risk measure of the terminal profit and loss (P&L), rather than by differentiating a pricing model. As a case study we sell a 30-day at-the-money European call option and train a network that, on each trading day, maps the current spot price, the time to maturity and the previously held position to the position to hold in the underlying. The training objective is the *entropic risk measure* of the P&L, whose optimal value coincides with the exponential-utility indifference price of the claim. Networks are trained on overlapping 31-day windows drawn from 18 historical S&P 500 sector ETFs and global-index trackers. We compare two architectures, a multilayer perceptron (MLP) and a gated recurrent unit (GRU), against a Black–Scholes (BS) delta benchmark. In the idealized frictionless world the learned strategy is expected to recover the BS delta and thus a delta-neutral portfolio, whereas in the presence of transaction costs it departs from delta neutrality, yielding a more stable hedge. Out of sample, both learned hedgers attain a lower entropic risk than the BS delta under proportional transaction costs.

1 Introduction

According to Investopedia, *hedging* means using financial instruments to offset the risk of adverse price movements. In the business of trading and selling derivatives it is the core of the day-to-day work of a desk: an institution that has sold an option is exposed to the future moves of the underlying asset and must hold an offsetting position so that gains and losses cancel as much as possible. The main tool of this practice is *delta hedging*, a strategy designed to remove the first-order exposure of a position to the price of the underlying.

To understand delta hedging it is necessary to recall what the *delta* of an option is. The delta measures how much the price of an option is expected to change when the price of the underlying asset moves by one unit. If an option has a delta of 0.5, then a \$10 increase in the underlying is expected to raise the option price by about \$5. The

delta is not static: it varies with the level of the underlying and as the option approaches maturity. By definition the underlying itself has a delta of one, so a position of 100 shares carries a portfolio delta of 100. A trader who shorts a call option with delta 0.5 must therefore hold 50 shares of the underlying to make the portfolio *delta-neutral*. Because the delta drifts as the spot moves and time passes, this neutrality is only instantaneous, and maintaining it requires the position in the underlying to be readjusted continuously.

Delta-neutral strategies are common both in pure hedging and in premium-selling strategies. In the latter, a trader sells options to collect the premium and then covers the directional (delta) risk by trading the underlying, so as not to be exposed to the direction of the market but rather to the *time decay* of the options. As maturity approaches, options lose value, and a premium seller who keeps the book delta-neutral profits from this decay as long as the underlying does not move too unfavourably. This is the textbook prescription of the Black–Scholes (BS) model, under which the delta is the partial derivative of the option price with respect to the spot and a continuously rebalanced delta hedge perfectly replicates the payoff.

This classical, model-driven recipe has limitations when it meets reality. The replication argument holds only in a frictionless and complete market, whereas in practice every rebalancing trade incurs transaction costs, bid–ask spreads and liquidity constraints that the standard Greek computation simply ignores. The Greeks also take no account of trading constraints such as limits on the desk’s delta, and for complex or exotic claims the Greeks themselves can be expensive to compute or numerically unstable. In short, the strategy that is optimal in the idealized model is not the strategy that is optimal once frictions are present.

This is why deep hedging could be an interesting tool for banks. A large institution runs an enormous book of derivatives, and the running cost of hedging it, that is, the cumulative transaction costs of rebalancing thousands of positions, is a direct and material drag on the desk’s profit and loss. A method that is model-independent, that optimizes the cost–risk trade-off over the whole portfolio rather than position by position, and whose computational cost scales with the number of hedging instruments rather than with the size of the book, can both automate hedging and reduce its cost. The original framework grew out of the investment-banking industry, and subsequent work has explicitly aimed at replacing Greek-based hedging on trading desks with learned strategies.

The same logic explains why deep hedging can also help a retail trader make money. A typical retail options strategy is exactly the premium-selling strategy described above: sell options, harvest the time decay, and stay delta-neutral so as not to bet on direction. On paper such a strategy is profitable, but in practice the frequent rebalancing it requires generates commissions and spreads that erode, and can even eliminate, the harvested premium. A hedging strategy that has *learned* to trade less aggressively when costs are high, accepting a slightly non-neutral delta in exchange for fewer trades, can preserve much more of the premium. For a cost-sensitive trader, deep hedging is therefore not only a tool for reducing risk but a way of turning a strategy that is theoretically attractive but destroyed by costs into one that is net profitable.

Motivated by this, our objective is to use neural networks and a fully data-driven approach to obtain an optimal hedging strategy that minimizes the risk of the P&L and that, under ideal conditions, is delta-neutral. At time t_0 we sell an at-the-money European call with 30 days to maturity, so that the strike is $K = S_0$. By selling the call we are exposed to the risk that the underlying rises, since at maturity we must pay the buyer the payoff $\max(S_{30} - K, 0)$. To cover this risk we hold the underlying and adjust the held

quantity every day, so that gains and losses on the underlying offset the call payoff as much as possible. Instead of computing the Black–Scholes delta each day, which would require committing to a specific price model, we train a network to find the strategy. At each day t the network observes a market state and outputs the position $\delta_t \in [0, 1]$ to hold on that day. For every trajectory of the underlying we compute the final P&L of the portfolio,

$$\text{PnL} = \sum_{t=0}^{T-1} \delta_t (S_{t+1} - S_t) - \sum_{t=0}^{T-1} c |\delta_t - \delta_{t-1}| S_t - \max(S_T - K, 0), \quad (1)$$

where $T = 30$ and c denotes the proportional transaction cost. The loss function is not an error with respect to a target value, since no “correct” position exists, but a risk measure on the P&L. Following Buehler et al. we use the *entropic risk measure*, whose optimal value is the exponential-utility indifference price of the claim (Section 5). In this way the network learns by itself the hedging strategy that makes the portfolio as stable as possible by penalizing losses exponentially. We expect that, without transaction costs, the resulting vector of positions $(\delta_0, \dots, \delta_{T-1})$ converges to the Black–Scholes delta and the portfolio is delta-neutral, whereas in the presence of transaction costs the network finds strategies that improve on the classical one by moving away from delta neutrality, more or less depending on c , in order to avoid overly frequent rebalancing. In deep hedging, therefore, delta neutrality is not imposed as in Black–Scholes but is a property that should emerge under ideal conditions. As inputs we use historical data of S&P 500 sector ETFs together with other global-index trackers that share similar market dynamics. We begin with a multilayer perceptron (MLP) and then compare it with a more expressive recurrent architecture, the gated recurrent unit (GRU) and then with a long short term memory (LSTM) architecture. We will use different benchmark in order to evaluate the performances of our model:

- Unhedged strategy
- Naive hedger
- Logistic regression
- Black-Scholes
- Black-Scholes with Leland sigma adjustment

The remainder of this report is organized as follows. Section 2 reviews the relevant literature on deep hedging. Section 3 describes the dataset and the construction of the trajectories. Sections 4 and 5 formalize the P&L and the entropic risk measure used as the training objective. Section 6.4 introduces the Black–Scholes baseline. Sections 7 and 8 present the two network architectures, and Section 11 reports the out-of-sample comparison.

2 Literature Review

Traditional option pricing and risk management heavily rely on the seminal framework established by Black and Scholes (1973) and Merton (1973). Their model assumes a

Table 1: The 18 instruments used to build the trajectories.

Group	Tickers
Broad market	SPY, QQQ, DIA, IWM
U.S. sectors	XLK, XLF, XLV, XLY, XLP, XLE, XLI, XLB, XLU
International	EWG, EWJ, EWL, FXI, EEM

friction-free market where an option can be perfectly replicated by continuously rebalancing a portfolio consisting of the underlying asset and a risk-free bond. However, in the presence of transaction costs, continuous rebalancing is unfeasible as it triggers infinite cumulative trading expenses, while discrete-time rebalancing introduces “hedging error” or residual risk. A first attempt to take into account market frictions such as fees in the option pricing was performed by Hayne E. Leland (1985) that introduced a correction of the historical sigma used in the BS model.

A further step ahead, Buehler et al. (2019) introduced the concept of Deep Hedging. By framing the portfolio replication problem as a sequential decision-making process under a specified convex risk measure, they demonstrated that neural networks can successfully optimize hedging strategies directly within incomplete and frictional market environments without requiring analytical tractability. This project builds upon their architecture by implementing a compact, efficient deep-learning model tailored to tackle proportional transaction costs, and follows the methodology illustrated in the practitioner study by Mazars on the application of deep learning to hedging.

3 Dataset

We collected real financial data using the `yfinance` library. Instead of a single stock, we downloaded daily prices for 18 different global indices and sector ETFs, listed in Table 1, which together span U.S. broad-market, sectoral and international exposures. The data covers the period from January 1, 2006 to December 31, 2025. Using multiple instruments ensures that our dataset contains different market regimes, including periods of high volatility, growth, and financial crises.

To train the deep-hedging model we need blocks of data that represent a 30-day trading period. We obtain them with a sliding-window technique: for each instrument the pipeline extracts every sequence of 31 consecutive trading days (from t_0 to t_{30}), sliding the window forward one day at a time. Each extracted path is normalized so that the initial price S_0 is always equal to 1.0; this normalization is crucial because it lets the network learn a hedging rule that is independent of the absolute nominal value of the instrument, and it makes the strike $K = S_0 = 1$ at-the-money by construction. The window length is $31 = T + 1$, i.e. the 30 trading steps plus the initial price.

After extracting the paths from all 18 instruments we combined them into a single matrix, which yields 90,000 trajectories of length 31. To avoid data leakage we first split the trajectories into a training set of 73,216 paths (chosen close to 80% of the sample and divisible by the batch size 512) and a test set of the remaining 16,784 paths, held out for the final out-of-sample evaluation against the Black–Scholes baseline and then, to mix market regimes, the trajectories are randomly shuffled. As a sanity check on the directionality of the sample, 60.9% of the training paths finish above their starting level over the 30-day horizon, consistent with the long-run upward drift of equity indices.

The deep-hedging framework does not try to predict the future direction of asset prices; instead it optimizes a tactical trading strategy based on the current state of the market. The network receives the mathematical inputs required by the theory of option replication, the current price S_t and the remaining time to maturity τ_t (the MLP additionally receives the previous holding δ_{t-1}), so no manual feature engineering is required for this task.

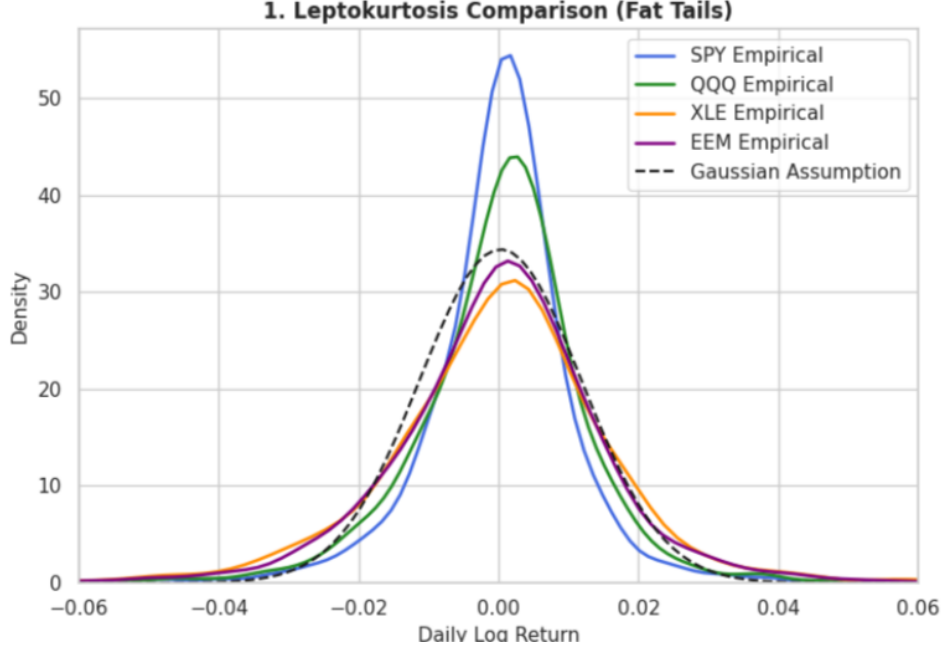


Figure 1: Leptokurtosis Comparison (Fat Tails). This plot compares the empirical log-returns of the asset basket against the theoretical Gaussian distribution used in the classic Black-Scholes model. The real market data shows a higher peak and significantly heavier tails. This clear evidence of leptokurtosis means that extreme market shocks happen more often in reality than a normal distribution predicts, which justifies using neural networks like MLP and GRU to capture these non-linear behaviours.

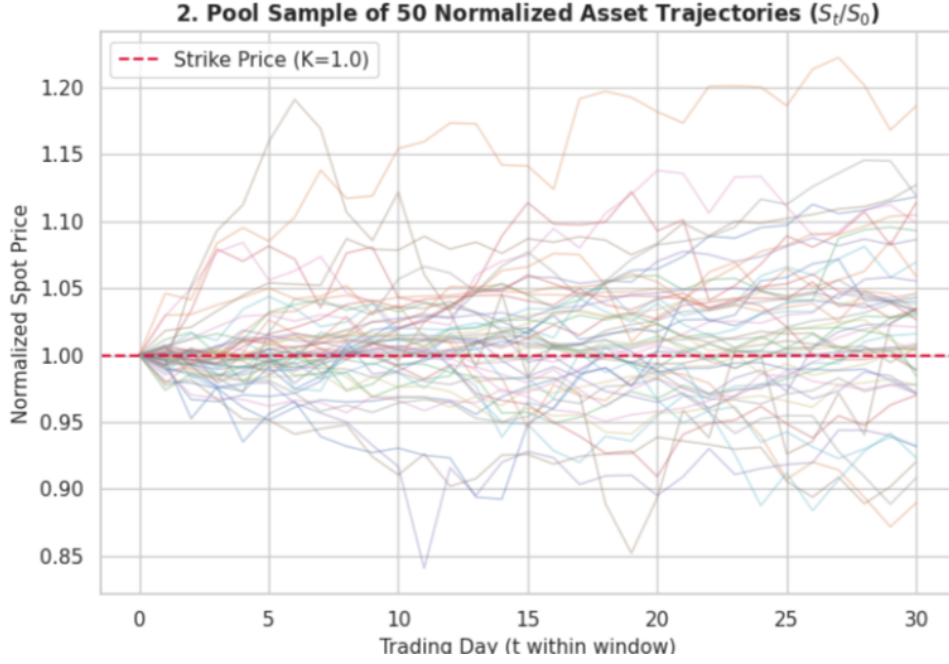


Figure 2: Pool Sample of 50 Normalized Asset Trajectories (S_t/S_0). This chart displays a random sample of 50 historical trajectories from the training dataset, normalized to a starting price of 1.0. The red dashed line represents the At-The-Money strike price. These paths expose the neural networks to real historical market regimes, including major financial crises, forcing the MLP and GRU models to learn robust hedging strategies under complex real-world conditions.

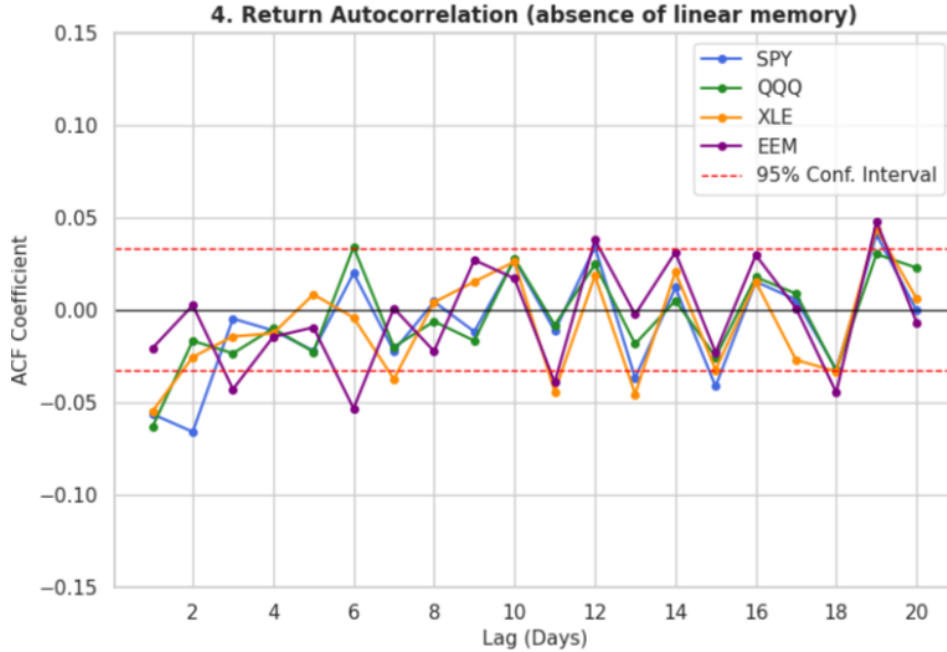


Figure 3: Return Autocorrelation (ACF). The autocorrelation function of the log-returns drops quickly and stays inside the 95% confidence interval from the very first lags. This indicates an absence of linear memory in asset returns, which aligns with the weak-form efficient market hypothesis. Because there is no strong serial correlation in the raw returns, a static model like the MLP might find it difficult to extract predictive patterns from past return sequences alone.

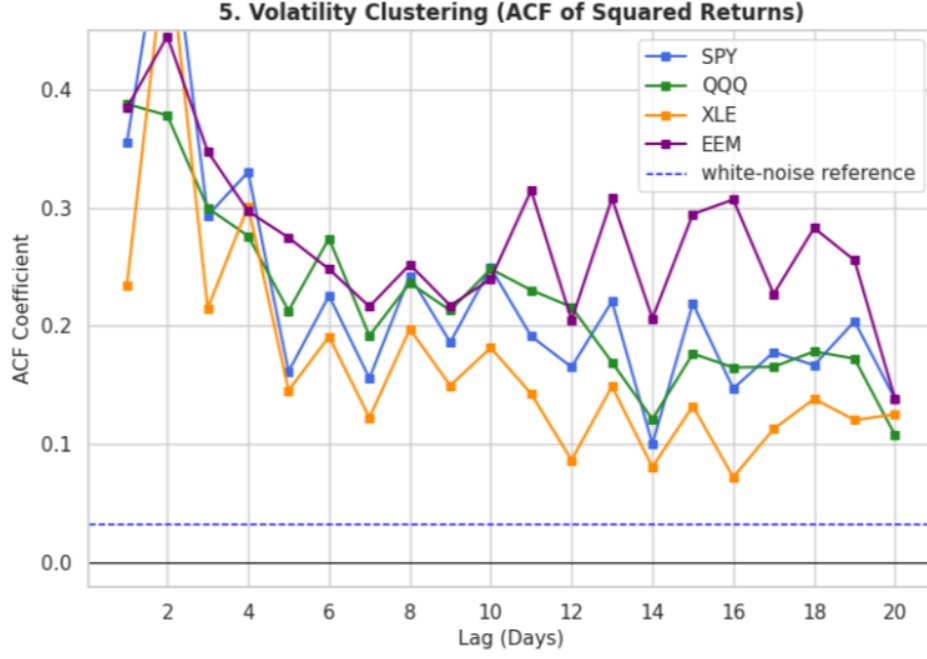


Figure 4: Volatility Clustering (ACF of Squared Returns). In contrast to the linear returns, the autocorrelation of squared returns shows strong and persistent positive coefficients well above the confidence threshold. This represents the phenomenon of volatility clustering, where high volatility periods tend to follow high volatility periods. Since volatility changes over time with clear memory, the GRU network has a structural advantage over MLP and Black-Scholes because its recurrent architecture is designed to track these temporal dependencies.

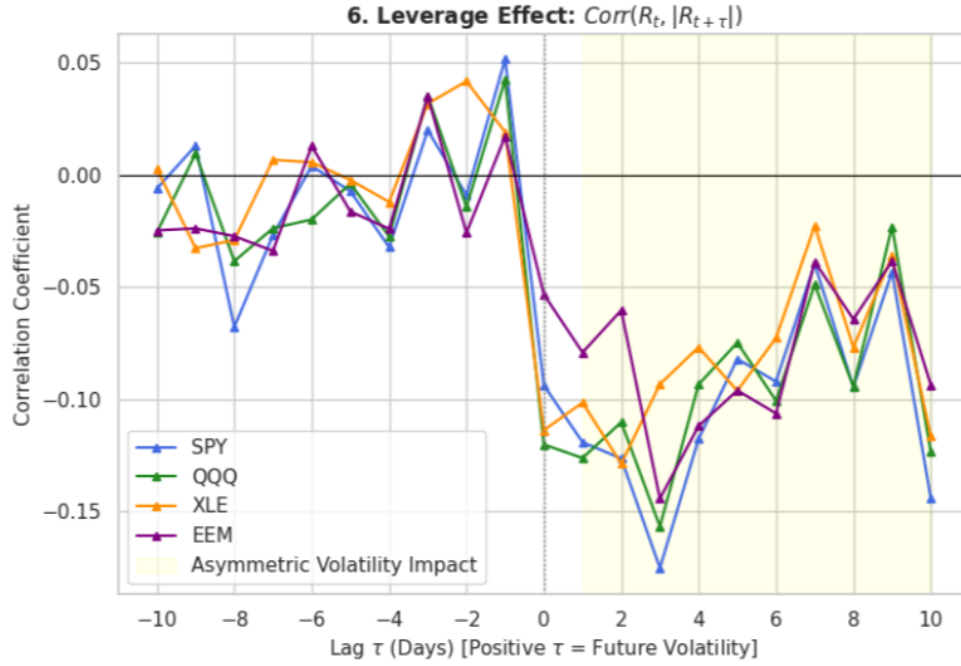


Figure 5: Leverage Effect across Sectors. This plot illustrates the cross-correlation between asset returns and future volatility. The significant negative correlation at positive lags shows a clear leverage effect, meaning that negative returns today tend to increase market volatility tomorrow. The standard Black-Scholes model assumes constant volatility and fails to capture this asymmetry, whereas deep learning models can learn this directional relationship between price drops and risk.

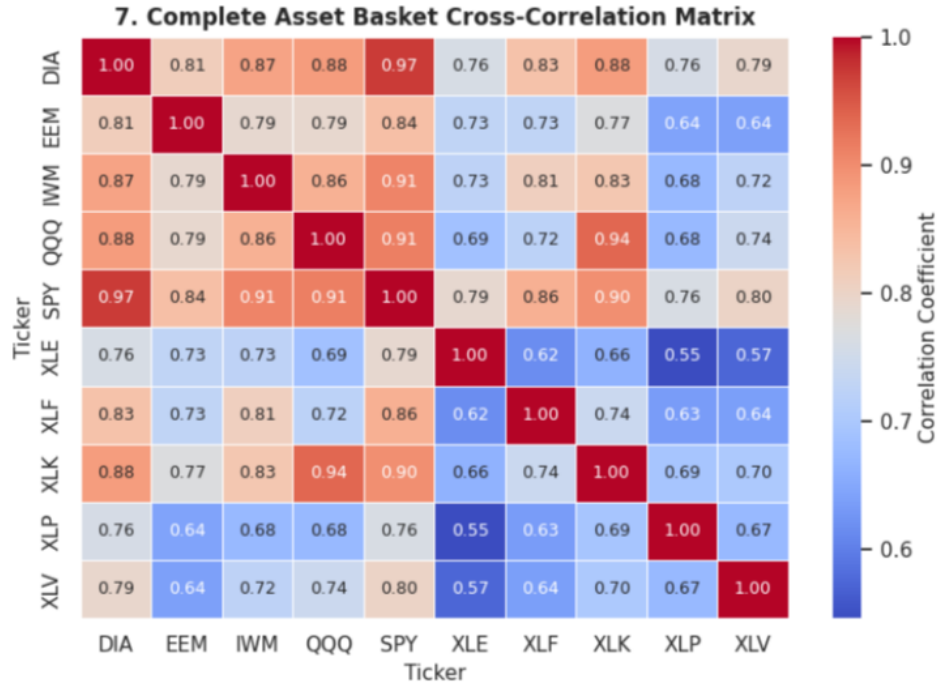


Figure 6: Complete Asset Basket Cross-Correlation Matrix. The heatmap shows the linear correlation coefficients among all ten ETFs included in the market basket. The high positive correlation between major indices like SPY, QQQ, and DIA indicates strong systemic connections across different sectors. For the deep hedging project, this interconnected basket provides a great framework to test how well the MLP and GRU models can handle joint market movements and asset co-movements.

Asset Ticker	Mean (μ)	Std Dev (σ)	Skewness	Excess Kurtosis
DIA	0.000276	0.01077	-0.477	15.527
EEM	0.000113	0.01682	-0.085	11.758
IWM	0.000255	0.01500	-0.356	11.454
QQQ	0.000468	0.01292	-0.592	7.869
SPY	0.000267	0.01159	-0.568	15.029
XLE	0.000045	0.01715	-0.626	11.737
XLF	0.000050	0.02033	0.443	21.310
XLK	0.000417	0.01326	-0.639	11.415
XLP	0.000281	0.00971	-0.916	23.218
XLV	0.000330	0.01119	-0.731	14.305

Table 2: Descriptive statistics summary for the analyzed assets.

The descriptive statistics table confirms that all assets in the basket deviate significantly from a standard normal distribution. Most tickers display negative skewness. Furthermore, every asset shows extremely high excess kurtosis, well above the Gaussian value of zero. This statistical summary provides strong evidence of fat tails and high risk in historical data, confirming that traditional Black-Scholes assumptions do not hold and reinforcing the need for flexible deep learning approaches like MLP and GRU.

4 Profit and Loss

We use the terminal Profit and Loss (P&L) to capture the monetary value of the strategy at the option’s maturity, measuring the hedging error net of real-world transaction costs. The terminal P&L is

$$\begin{aligned}
\text{PnL}_T = & -\max(S_T - K, 0) \\
& + \sum_{t=0}^{T-1} \delta_t (S_{t+1} - S_t) \\
& - \sum_{t=0}^{T-1} c |\delta_t - \delta_{t-1}| S_t
\end{aligned} \tag{2}$$

subject to the boundary condition $\delta_{-1} = 0$, representing the empty starting position at $t = 0$. The individual components are defined as follows:

- PnL_T : the terminal P&L of the hedging portfolio at maturity T ;
- $\delta_t \in [0, 1]$: the hedging position (number of shares) chosen by the network at time t ;
- S_t, S_{t+1} : the underlying prices at times t and $t + 1$;
- $(S_{t+1} - S_t)$: the price change of the underlying over the daily step;
- c : the proportional transaction-cost coefficient ($c = 0.001$, i.e. 10 basis points, in all experiments);

- $|\delta_t - \delta_{t-1}|$: the size of the trade executed at the rebalancing time t ;
- $\max(S_T - K, 0)$: the payoff of the short European call at maturity, paid out by the hedger.

Two remarks connect Eq. (2) to the economics of the trade. First, the sign of the payoff term encodes that we are *short* the call: the hedger owes $\max(S_T - K, 0)$ at maturity and holds $\delta_t \geq 0$ shares to offset it, which is consistent with a positive Black–Scholes call delta. Second, Eq. (2) does *not* include the premium received at inception for selling the option; consequently the mean P&L of any strategy is shifted downward by approximately the option’s fair value. The premium can be recovered as the optimal value of the entropic objective (Section 5): adding it back recenters the P&L distribution near zero and turns the comparison into a like-for-like statement about risk.

5 Entropic Loss

The entropic risk measure serves as the objective function of our deep-hedging framework. As shown by Buehler, Gonon, Teichmann and Wood (2019), optimizing a hedging strategy under this measure is equivalent to computing the exponential-utility indifference price of the derivative. Let X denote the terminal P&L (gains counted positively) and let $\lambda > 0$ be the risk-aversion parameter. The entropic risk measure is

$$\rho(X) = \frac{1}{\lambda} \log \mathbb{E}[\exp(-\lambda X)]. \quad (3)$$

Financially, this imposes an exponential penalty on negative realizations of the P&L. The parameter λ controls the severity of the penalty: a higher λ reflects a more risk-averse profile, forcing the network to prioritize the minimization of extreme tail losses over average gains; in our experiments we use $\lambda = 10$. Being convex, the measure provides a well-behaved gradient landscape for stochastic optimization. It is also *cash-invariant*, $\rho(X + a) = \rho(X) - a$, which is precisely the property that lets us read the indifference price off the optimum: at the optimal strategy the premium that makes the hedger indifferent between selling-and-hedging and doing nothing is $p = \rho(X^*)$.

In our TensorFlow implementation the loss is computed on the terminal P&L over each mini-batch. The P&L tensor is first cast to `float32` and reshaped to a flat vector, so that the same routine can be used both inside the training graph and on the flattened test arrays. The argument of the exponential is $-\lambda X$, and the log-mean-exponential is evaluated with the numerically stable `tf.math.reduce_logsumexp`, from which $\log N$ is subtracted to turn the log-sum into a log-mean over the batch of size N :

$$\rho(X) \approx \frac{1}{\lambda} \left[\log \sum_{i=1}^N \exp(-\lambda X_i) - \log N \right]. \quad (4)$$

Finally the value is scaled by $1/\lambda$ to obtain the scalar loss, which is differentiated by the optimizer (Adam) to update the network weights, in the recurrent case through Backpropagation Through Time. A single, shared `entropic_loss` routine handles tensors of shape $(N,)$, $(N, 1)$ or (N, k) via an internal reshape, so the objective used in training and the metric reported at evaluation are guaranteed to be identical.

6 Baseline Model

6.1 Unhedged

We compute the profit and loss without applying any hedging strategy. In this way the operator is totally exposed to the risk and will face losses equal to the premium that has to pay if the option is in money

6.2 Naive hedger

This model will implement the following hedging strategy. When the price of the underlying is above the strike price the delta is predicted to be equal to one and to zero otherwise. This strategy will be extremely expensive if the price will fluctuate around the strike price for the whole window.

$$\Delta_t = \begin{cases} 1 & \text{if } S_t > K \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

6.3 Logistic regression

The model shares exactly the same input features as the Multi-Layer Perceptron (MLP), receiving the state of the market and the portfolio at time t as input through the following triplet of features: Δ_{t-1} : the position (delta) held in the previous period. S_t : the current price of the underlying asset (or its moneyness). τ_t : the time remaining to maturity. Unlike deep architectures, the logistic regression processes this information through a single linear layer (with no hidden layers) followed by a sigmoid activation function, which maps the output into the $[0, 1]$ interval.

$$\Delta_t(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_{\Delta} - 1x + \beta_{maturity}x_0\beta_{price}x)}} \quad (6)$$

6.4 Black-Scholes

We use the Black-Scholes-Merton (BSM) framework as the baseline against which the learned hedgers are evaluated, since it is the standard reference in quantitative finance. BSM computes the hedge ratio (delta) of the option, the sensitivity of its price to changes in the underlying, and a portfolio that holds this many shares neutralizes the first-order directional risk. The inputs are the current price, the strike, the remaining time to maturity, the volatility, and the risk-free rate. In our experiments the baseline uses a constant volatility $\sigma = 0.20$ and a risk-free rate $r = 0$, with the time to maturity expressed in years as $\tau = (T - t)/252$.

The model evaluates the moneyness of the option through the term

$$d_1 = \frac{\log(S/K) + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}, \quad \tau = \frac{T - t}{252}, \quad (7)$$

and the call delta is the cumulative standard normal of d_1 ,

$$\Delta_{\text{call}} = \Phi(d_1). \quad (8)$$

Worked example. Consider writing an ATM call at $t = 0$ with $S = K = 100$, $T = 30$ days, $\sigma = 0.20$, $r = 0.04$ and a transaction fee of 0.1%, so that $\tau = 30/252 \approx 0.1190$ years. Then

$$d_1 = \frac{\log(1) + \left(0.04 + \frac{0.2^2}{2}\right) (0.1190)}{0.2 \sqrt{0.1190}} \approx 0.1035,$$

and $\Delta_{\text{call}} = \Phi(0.1035) \approx 0.5412$, so the short-call position requires holding about 0.5412 shares immediately. If on the next day the spot rises to $S = 102$, the recomputed hedge ratio expands to roughly $\Delta_{\text{call}} \approx 0.6531$, forcing the purchase of an additional $0.6531 - 0.5412 = 0.1119$ shares. That single rebalancing trade costs

$$|\Delta_t - \Delta_{t-1}| \cdot S \cdot c = |0.6531 - 0.5412| \cdot 102 \cdot 0.001 \approx 0.0114$$

in fees. Because the BS strategy rebalances purely to track the price, it incurs this penalty at every one of the 30 steps. This compounding cost, together with the assumption of frictionless, normally distributed returns, is the structural weakness of the baseline that a cost-aware learned hedger can exploit.

In the out-of-sample experiment the baseline is computed exactly as above on each normalized test path, with $K = 1$. Two caveats are worth stating explicitly. First, the baseline uses a single, fixed volatility $\sigma = 0.20$ across all 18 instruments, whereas a desk would estimate σ per instrument; the learned hedgers, by contrast, never observe which instrument generated a path. Second, the network receives the time to maturity as a normalized fraction $\tau_t = (T - t)/T \in [1/T, 1]$, while the BS delta uses the annualized $\tau = (T - t)/252$; the two clocks differ, and only the BS leg is tied to the annual convention through σ .

6.5 Leland adjustment

The Leland correction, introduced by economist Hayne E. Leland in 1985, is a mathematical adjustment applied to the standard Black-Scholes model to incorporate the impact of proportional transaction costs.

In the original theoretical framework of Black and Scholes, the market is frictionless, and the hedger rebalances the portfolio continuously. In the presence of transaction fees, continuous rebalancing would lead to infinite transaction costs, resulting in a certain loss. Leland resolves this paradox by assuming portfolio rebalancing occurs at regular discrete intervals (e.g., daily) and by modifying the volatility parameter.

The Modified Volatility Formula

Leland demonstrated that it is possible to compensate for trading costs by replacing the classic volatility σ with a modified volatility $\hat{\sigma}$, calculated using the following expression:

$$\hat{\sigma} = \sigma \sqrt{1 + \sqrt{\frac{2}{\pi}} \frac{c}{\sigma \sqrt{\Delta t}}} \quad (9)$$

Where:

- σ is the original volatility (historical or implied) of the underlying asset.

Table 3: MLP hedger architecture (1,217 trainable parameters).

Layer	Units	Activation
Input	3	—
Dense	16	ReLU
Dense	16	ReLU
Dense	1	Sigmoid

- c is the proportional transaction cost (e.g., the fee expressed as a percentage of the traded volume).
- Δt is the time interval between two consecutive rebalances (e.g., $1/252$ for daily rebalancing).

7 Multilayer Perceptron

The first network we train is an MLP. We start from this base architecture before moving to a recurrent structure; the choice of an MLP as a deep hedger is itself supported by Buehler et al. (2019). We train a single network for the entire option lifetime, so the same model decides the hedge each day, an extension explicitly suggested in the original paper. The architecture is summarized in Table 3.

The network takes three inputs:

- Price of the underlying at time t (S_t): determines the option’s moneyness, which drives directional exposure, and scales the absolute monetary impact of the proportional transaction costs.
- Fraction of time to maturity (τ_t): accounts for the evolution of the option’s risk as it approaches expiry.
- Current portfolio position (δ_{t-1}): lets the network reason about the cost of changing its holding, which is essential under transaction costs.

The input layer is fully connected to two hidden layers of 16 ReLU neurons each (ReLU as suggested by Buehler et al.). The output layer has a single neuron, since there is one option and one underlying, with a *sigmoid* activation that forces the hedge into the bounded interval $[0, 1]$, preventing the hedger from short-selling (output below zero) or leveraging (output above one) the underlying. Empirically the sigmoid output also minimized the mean of the loss distribution among the bounding choices we tried.

The training cycle reflects the fact that the loss is defined only at maturity, not at every intermediate day. Within each step the loop iterates over the 30 trading days, at each day building the state $(\delta_{t-1}, S_t, \tau_t)$, querying the MLP for the new position, accumulating the hedging gain and the transaction cost into the running P&L, and carrying the position forward. Only after the payoff is subtracted at maturity is the entropic loss of Eq. (4) evaluated and the gradient taken. Training uses the Adam optimizer (learning rate 10^{-3}) for 30 epochs over 143 mini-batches of 512 paths.

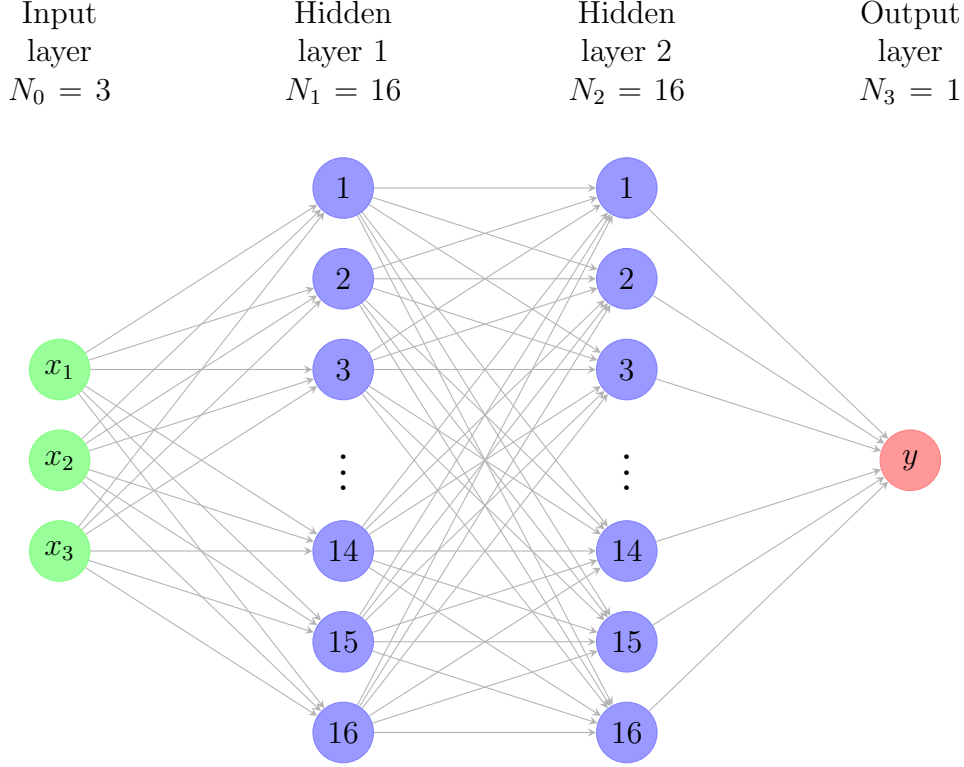


Figure 7: MLP architecture diagram.

8 Gated Recurrent Unit

The second architecture is a Gated Recurrent Unit (GRU). The MLP treats each trading day as an isolated event and therefore cannot capture the temporal structure of the path. To address this we move to a recurrent architecture, which is designed to process sequential data and is, in theory, better suited to the deep-hedging problem because the hedge at step t can depend on the whole history $S_1, \dots, S_t$, a property that matters as soon as transaction costs make the strategy path-dependent.

Standard RNNs suffer from the vanishing-gradient problem when information must be propagated through many steps. The GRU mitigates this with internal gates that decide how much past information to keep and how much to forget:

- Update gate: controls how much of the past hidden state is carried to the current step, helping the network maintain memory.
- Reset gate: controls how much past information is discarded, letting the model drop stale context when the market regime changes abruptly.

The architecture is summarized in Table 5. A key difference from the MLP is the *input*: the GRU receives only two features per step, the price S_t and the fraction of time to maturity τ_t . The previous position δ_{t-1} is *not* fed in explicitly, because the GRU's hidden state already carries the memory of past decisions, making it the recurrent counterpart of the MLP's third input.

The GRU processes the full 30-step path in a single forward pass and, with `return_sequences = True`, produces a delta at each step as a (batch, 30, 1) tensor. The previous-position vector needed for the transaction-cost term is obtained by shifting this sequence by one step (with $\delta_{-1} = 0$), and the entire P&L of Eq. (2) is then evaluated in one

Table 4: GRU hedger architecture (3,489 trainable parameters).

Layer	Shape / Units	Activation
Input	$(T, 2) = (S_t, \tau_t)$	—
GRU	64 (return sequences)	tanh / sigmoid gates
Dense	1	Sigmoid

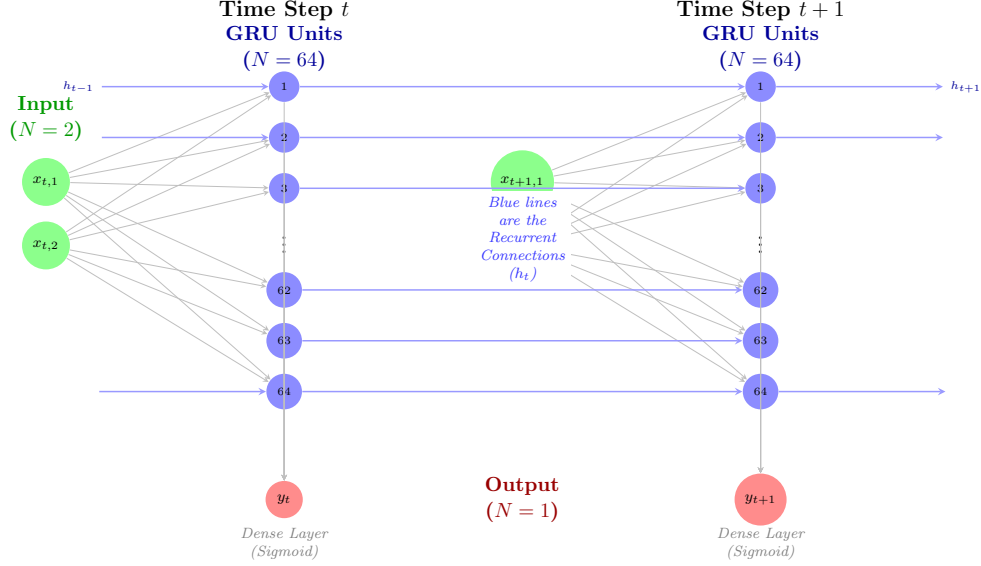


Figure 8: Diagram of the GRU architecture unrolled in time. The recurrent units transmit the hidden state h_t to the next time step, acting as the memory carrier for the hedging position.

vectorized operation, with no Python loop over time. As in the MLP, a final single-neuron sigmoid layer maps the GRU output to a position in $[0, 1]$, interpreted as the fraction of the underlying to hold. The training objective, optimizer and schedule are identical to the MLP (entropic loss with $\lambda = 200$, Adam at 10^{-3} , 30 epochs); the errors are backpropagated through time to adjust all weights and gate parameters. This setup lets us test whether adding temporal memory to the hedger yields an economic advantage in the presence of real market frictions.

9 LSTM

The third architecture implemented is the Long Short Term Memory. The LSTM is an advanced recurrent neural network designed to overcome the vanishing gradient problem. It efficiently captures sequential dependencies, making it highly effective for financial time-series modeling.

Dual Memory States

- Long-Term Memory (Cell State, C_t): The core memory that acts as a conveyor belt, preserving critical information over long sequences with minimal modification.
- Short-Term Memory (Hidden State, h_t): The network's output for the current time step, providing immediate context and feeding into the next step.

Table 5: LSTM hedger architecture (3,489 trainable parameters).

Layer	Shape / Units	Activation
Input	$(T, 2) = (S_t, \tau_t)$	—
LSTM	16 (return sequences)	tanh / sigmoid gates
Dense	1	Sigmoid

The Gating Mechanism LSTMs control information flow using three sigmoid-based gates (which output values between 0 and 1, meaning "block" to "pass through"):

1. Forget Gate (f_t) Decides which past information to discard from the long-term memory.

2. Input Gate (i_t) Determines what new information to store. The sigmoid layer (i_t) selects values to update, while a tanh layer creates new candidate values ($\tilde{C}_t$). These combine to form the new cell state

3. Output Gate (o_t) Filters the cell state to generate the new hidden state (h_t), which serves as the immediate prediction. A difference with respect the GRU structure is that here have 2 memory cells while in the GRU we had just one. This is a more complex structure which is more costly to be trained but can take better into account the long term memory. The graph of the model is represented in the the figure 9 while the architecture is summarized in Table 5. The train procedure is the sam used for the GRU structure.

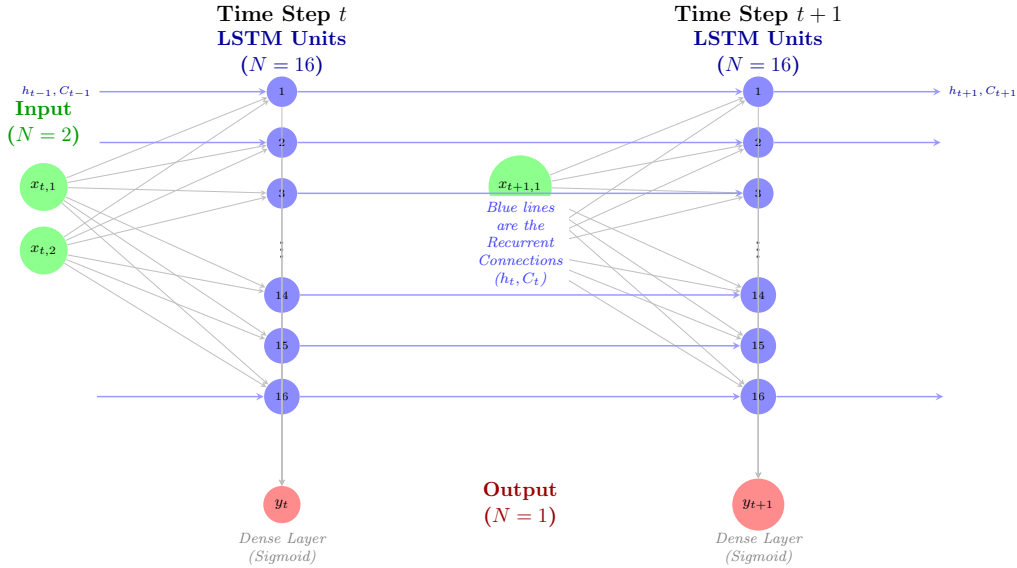


Figure 9: Diagram of the LSTM architecture unrolled in time. The recurrent units transmit the hidden state h_t and cell state C_t to the next time step, acting as the memory carrier for the hedging position.

10 Models selection

In order to perform a proper model selection for each neural network architecture we performed a grid search evaluating the performances on a fixed validation set (from "2020-01-01" to "2021-12-31"). The hyperparameters that we optimized are: the number of

epochs, the learning rate and the width of layers. The parameter search space was defined across two primary dimensions:

- Learning Rate: 3e-4, 1e-3, and 3e-3
- Hidden Layer Width: 16, 32, and 64 nodes

For each architecture, this predefined grid resulted in the initialization and training of 9 distinct model configurations.

To mitigate overfitting and dynamically determine the optimal training duration (the third hyperparameter: epochs), an early stopping mechanism was implemented rather than relying on a fixed epoch count. Each of the 9 models per architecture was allowed to train for a maximum of 30 epochs, with a patience parameter strictly set to 5. Consequently, the training routine was halted if the objective metric—the Entropic Loss evaluated on the static validation set—failed to demonstrate improvement for 5 consecutive epochs.

The results of the model selection are summarized in the following table

Architecture	Width	Learning Rate	Epochs
MLP	16	0.003	12
GRU	64	0.0003	11
LSTM	16	0.003	3

Table 6: Selected optimal hyperparameters for each neural network architecture.

11 Results

We evaluate the three learned hedgers and the baselines benchmark out of sample on the held-out test paths described in Section 3. The configuration is the same we used throughout the report: a proportional transaction cost of $c = 10$ bps ($c = 0.001$), risk aversion $\lambda = 10$, maturity $T = 30$, and a baseline volatility (historical standard deviation) $\sigma = 0.2265$ and a leland adjusted sigma $\hat{\sigma}^2 = 0.2328$ with $r = 0.045$. The loss function is the entropic risk ρ of the terminal P&L, Eq. (4), which is also the training objective; alongside it we report the average turnover (the total volume traded over the life of the hedge), and, where the shape of the risk profile is at issue, the mean, standard deviation and 95% expected shortfall ES_{95} (the average of the worst 5% of P&L outcomes, reported as a positive loss). For all three quantities a lower value is the better outcome. The premium collected at inception is excluded from the P&L, so every mean is shifted downward by approximately the option’s fair value.

11.1 Evaluation on test set with standard environment

Table 7 summarizes the out-of-sample performance of the evaluated hedging strategies. The neural network architectures (MLP, GRU, and LSTM) consistently outperform traditional mathematical models (Black-Scholes and Leland) in minimizing the objective Entropic Loss, effectively balancing the trade-off between hedging error and transaction costs. Notably, the MLP achieves the lowest Entropic Loss (0.0258). While the Logistic

regression serves as a strong, yet slightly inferior, learned baseline, with respect the neural network models it present the lowest PnL but the highest standard deviation, (highlighting a more volatile performances). As expected, the unhedged and naive strategies exhibit the highest risk profiles and worst overall performance across the evaluated metrics.

Model	Mean PnL	Entropic Loss	Std. Dev.	CVaR (95%)
MLP	-0.0248	0.0258	0.0133	0.0624
GRU	-0.0248	0.0262	0.0161	0.0687
LSTM	-0.0246	0.0261	0.0167	0.0714
Logistic	-0.0243	0.0265	0.0195	0.0799
Black-Scholes	-0.0280	0.0287	0.0118	0.0600
Leland	-0.0279	0.0287	0.0118	0.0597
Naive	-0.0282	0.0318	0.0248	0.0986
Unhedged	-0.0300	0.0399	0.0390	0.1410

Table 7: Out-of-sample performance metrics for the evaluated hedging models.

In the following figure we can see the distribution of the PnL for the various models. Observing the naive distribution we can observe that it's similar to a uniform distribution highlighting the randomness of the PnL distribution without applying any hedging strategy.

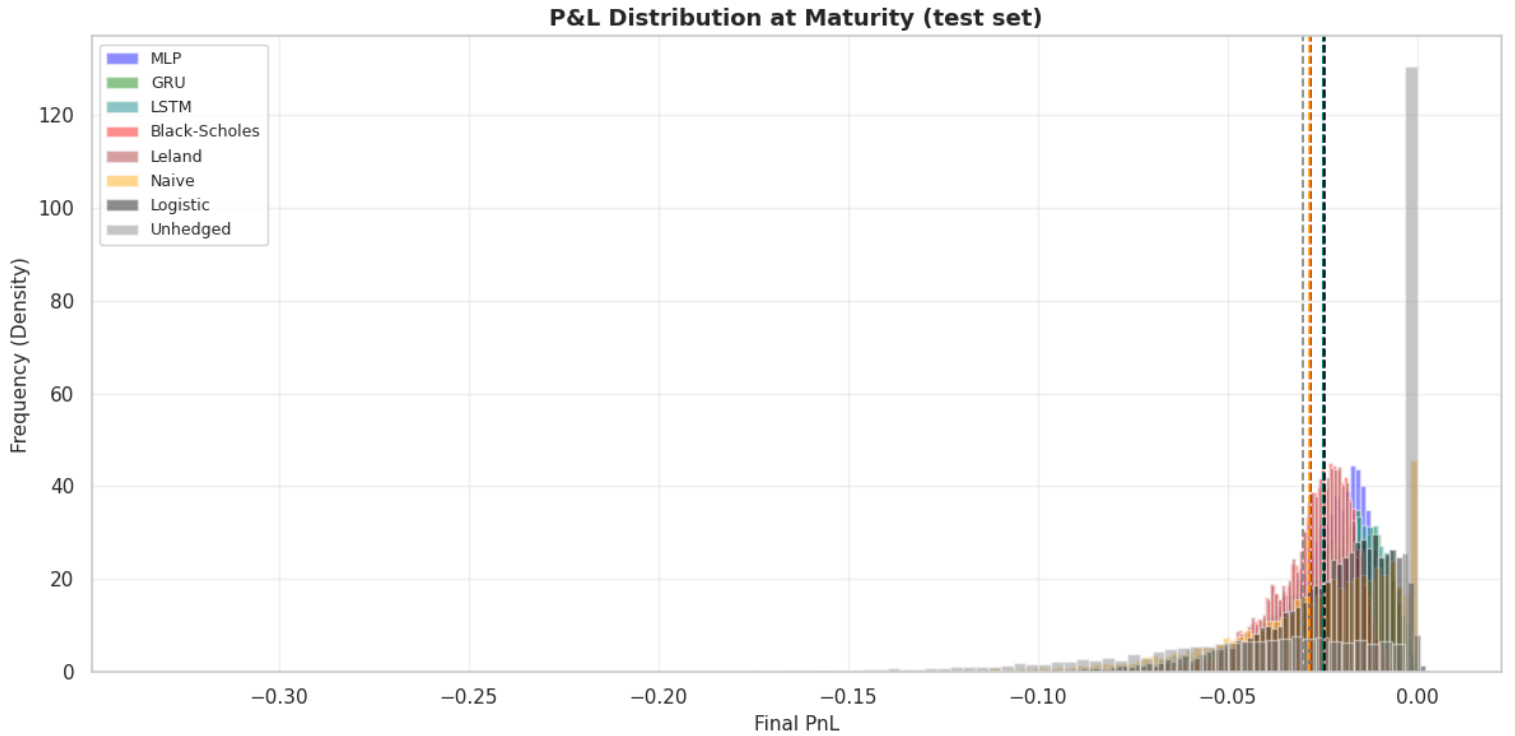


Figure 10: PnL distributions

Because the standard test windows overlap by 29 days (30-day slices with a 1-day stride), treating them as independent observations severely understates the sampling uncertainty. To address this structural autocorrelation, we apply a bootstrapping procedure

on a strictly non-overlapping dataset consisting of $n = 320$ disjoint windows. This approach allows us to compute an honest 90% confidence interval for the Entropic Loss across all strategies. The results, reported in Table 8, confirm the robustness of the deep hedging models: the MLP, GRU, and LSTM strictly dominate traditional baselines, exhibiting lower mean losses and tighter confidence bounds. Black-Scholes and Leland show structurally higher costs but a tighter confidence interval, while the Unhedged position collapses under extreme variance.

Model	Entropic Loss	90% Confidence Interval
MLP	0.02722	[0.02565, 0.02882]
GRU	0.02729	[0.02526, 0.02915]
LSTM	0.02747	[0.02545, 0.02942]
Logistic	0.02880	[0.02624, 0.03116]
Leland	0.02929	[0.02806, 0.03047]
Black-Scholes	0.02931	[0.02807, 0.03049]
Naive	0.03288	[0.03001, 0.03581]
Unhedged	0.04172	[0.03534, 0.04790]

Table 8: Bootstrapped Entropic Loss and 90% confidence intervals, computed using $n = 320$ disjoint windows to eliminate overlap bias.

We can see graphically the difference between the model performances in the following image

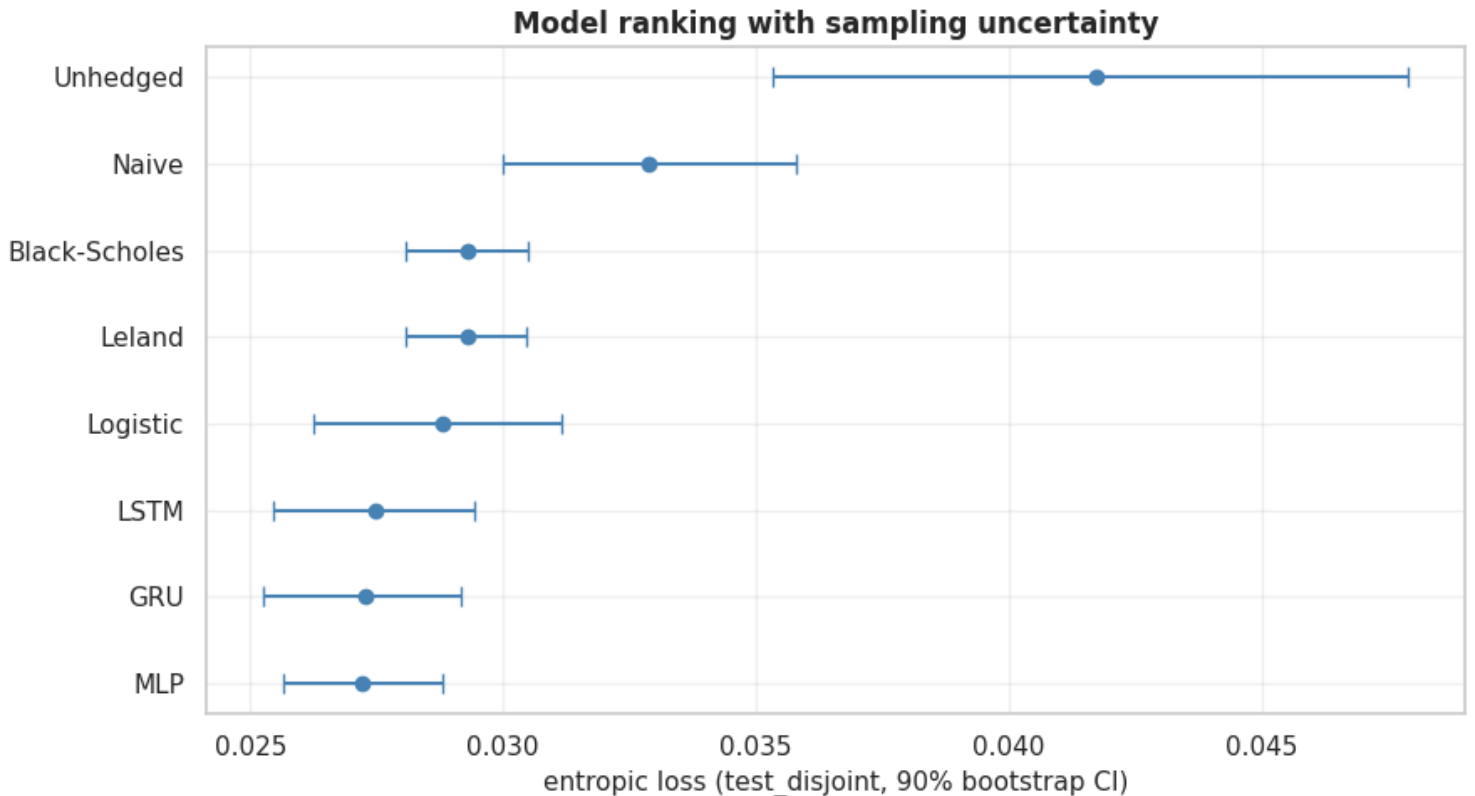


Figure 11: PnL distributions

12 Analyze the behavior of the models

In order to understand how a model behave and in which way take its hedging strategy we will take a single path (from the test set), run the models on it and analyses the difference strategies and see for each time step how the delta evolves. Then we will analyze for each time step how the models behave on average on the whole test set.

12.1 Single Path

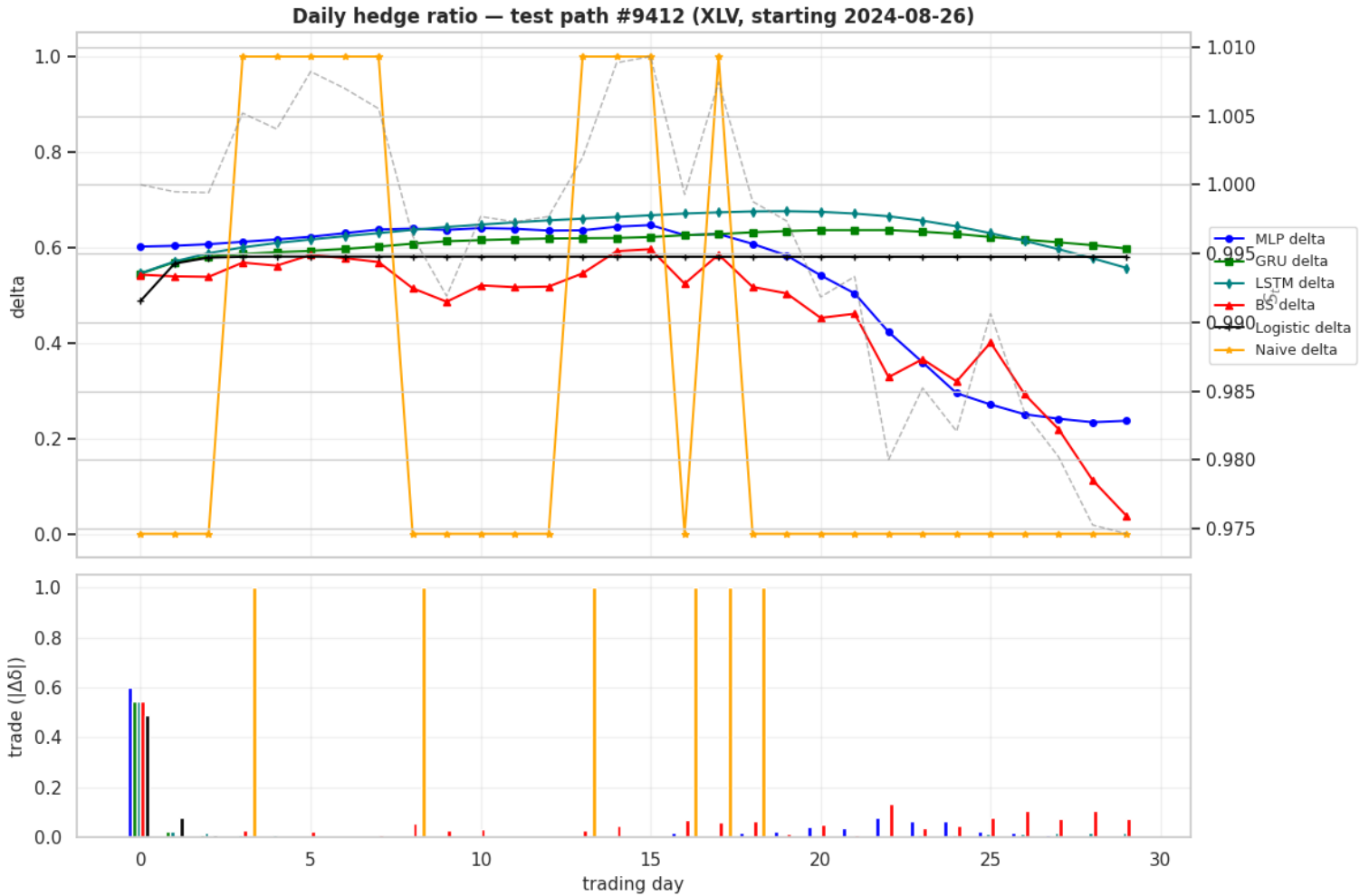


Figure 12: PnL distributions

- **The Unhedged** baseline is, by definition, a straight flat line at zero.
- **The Naive hedger** exhibits a step-like behavior by construction, aggressively jumping between 0 and 1.
- **The Logistic regression** computes a delta in the initial time steps that remains completely constant until maturity. Due to its simple single-layer structure, the model overestimates the impact of transaction costs, failing to adapt to market movements and the risk of the final payoff.

- **The Black-Scholes (BS)** model is highly sensitive to market movements. As the underlying price (dashed grey line) starts a downward trend below the strike price, the BS delta drops drastically, generating continuous adjustments without accounting for transaction costs.
- **The MLP** model tracks the general trend of the BS delta but in a much smoother manner. By successfully incorporating the transaction cost penalty, it avoids over-reacting to daily market noise while still adjusting the position.
- **The recurrent architectures (GRU and LSTM)** exhibit similar behavior and are thus analyzed together. They display a much more static profile compared to the MLP, indicating that recurrent structures heavily penalize trading activity in this specific path to save on commissions. Between the two, the LSTM remains slightly more dynamic and responsive to market behavior than the GRU.

In summary, the Naive strategy is totally blind to transaction costs and generates the largest trading volume (huge spikes in the lower panel). Similarly, the BS model ignores transaction costs and generates a high frequency of trades. Conversely, the Logistic regression over-penalizes costs, resulting in the lowest number of transactions (effectively zero after initialization). Among the deep learning models, the MLP dynamically balances the trade-off, generating the highest amount of transactions among the neural networks, whereas the recurrent structures generate significantly fewer trades.

12.2 Average behavior

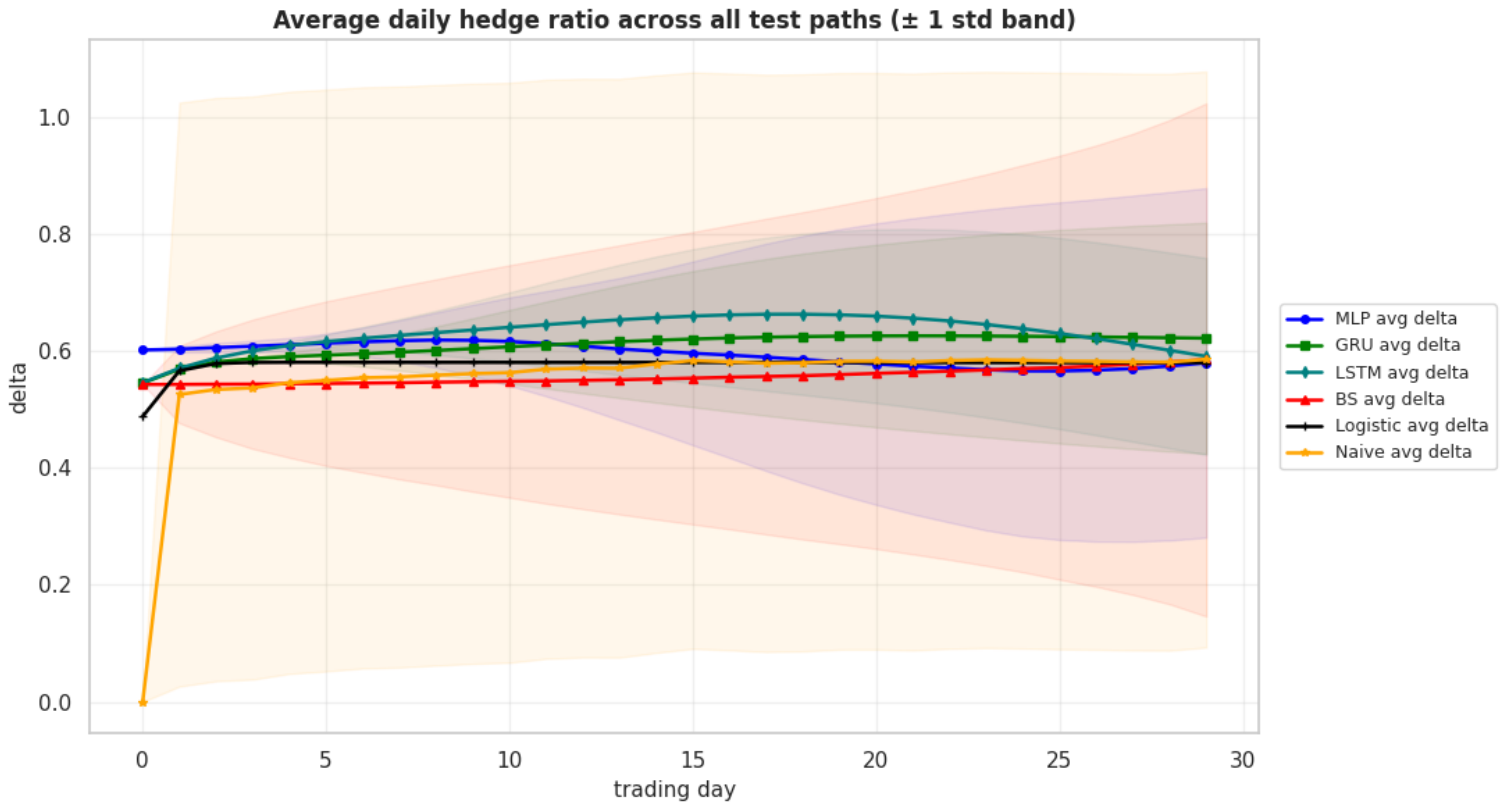


Figure 13: PnL distributions

The graph illustrates the evolution of the average hedge ratio (delta) over the 30 trading days, calculated across all test paths. The shaded areas represent the standard deviation bands (± 1 std), providing a visual measure of the responsiveness and variance of each model as market scenarios vary. From the visual analysis, the following dynamics emerge:

- **Naive Model** : It exhibits the most extreme variance by far, with the standard deviation band covering almost the entire $[0, 1]$ interval.
- **Logistic Regression (black line)**: It visually confirms its structural rigidity. After assuming the position on the first day, the average hedge remains perfectly horizontal and constant until maturity, with negligible variance. This reaffirms the linear model's inability to dynamically adapt to the stochastic evolution of the underlying asset.
- **Black-Scholes (red line)**: Its standard deviation band progressively widens as maturity approaches, reflecting the explosion of Gamma and the model's consequently higher sensitivity to final price movements.
- **Deep Learning Models (MLP, GRU, LSTM)**: On average, the neural networks tend to maintain a slightly higher hedge position compared to the Black-Scholes model (between 0.60 and 0.65).
 - The **MLP (blue line)** starts from a higher initial delta (0.60) compared to the other models, which decrease by approaching the maturity. The standard deviation band confirms an higher reactivity to the market behaviour.
 - The **Recurrent Networks (GRU and LSTM)** begin their hedge aligned with the Black-Scholes value (0.55), but gradually increase their position during the central days of the time window.

13 Out-of-sample switch enviroment condition

In this section we will analyze how the performances changes by changing environment condition which are defined by: the risk aversion, the transaction cost window length. Retraining happens for every neural/learned model (MLP, GRU, LSTM, Logistic) at every grid point — Black-Scholes, Leland, Naive and Unhedged need no retraining since they have no learned parameters. Will be analyzed how change the entropic loss and the turnover (total absolute change in delta over the path)

13.1 Switching transaction cost

The ex-ante expectation on how the behaves evolves by increasing the fees is an increase of the performance of the deep models with respect the BS. While by approaching to zero fees the deep models will converge to the BS strategy. However we expect an increase of the entropic loss for all the models while we expect a decrease in turnover for the deep architecture and for the logistic regression.

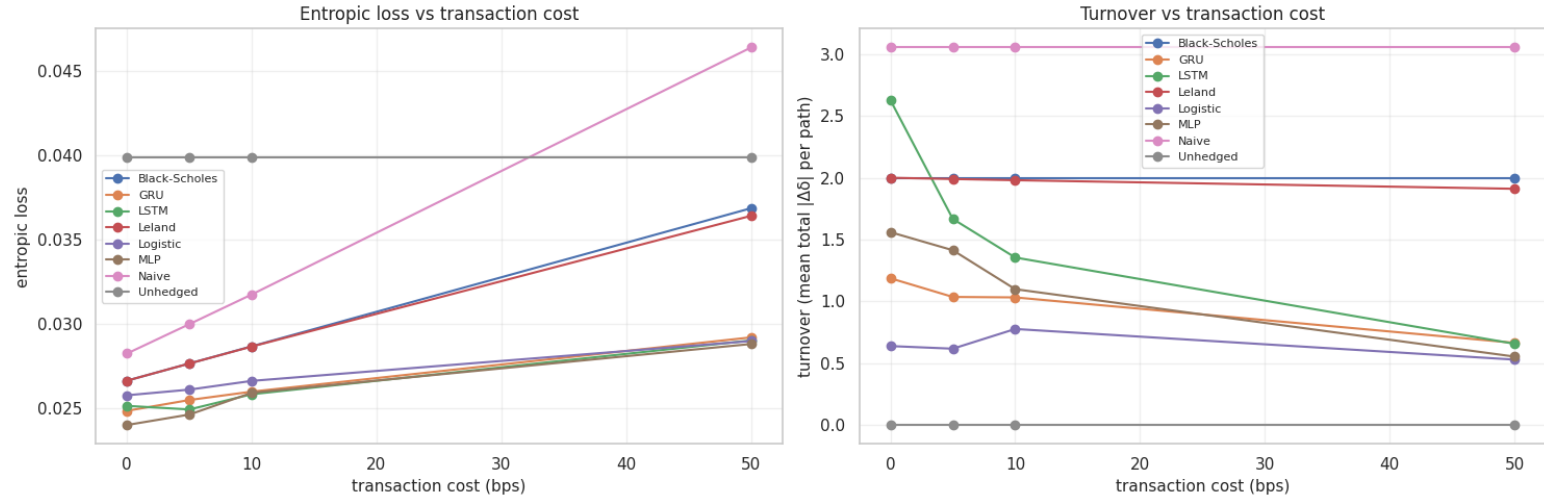


Figure 14: PnL distributions

As expected, the introduction of higher transaction costs systematically degrades the performance of all active hedging strategies, pushing the Entropic Loss upward.

The Unhedged portfolio remains mathematically constant, acting as an absolute upper bound for the risk. Across all tested fee levels, the deep learning architectures (MLP, GRU, and LSTM) consistently dominate the traditional baselines, maintaining a strictly lower Entropic Loss. This demonstrates their superior ability to optimize the risk-reward profile even in highly frictional environments. The convergence of the logistic regression with more complex models is explained by the statical behaviour that adopt the models with the increase of the fees that converge versus the statical behavior of the logistic.

fee_bps	B-S	GRU	LSTM	Leland	Logistic	MLP	Naive	Unhedged
0.0	0.02665	0.02486	0.02515	0.02665	0.02578	0.02401	0.02826	0.03991
5.0	0.02766	0.02550	0.02494	0.02766	0.02611	0.02463	0.03000	0.03991
10.0	0.02868	0.02599	0.02584	0.02867	0.02663	0.02593	0.03176	0.03991
50.0	0.03688	0.02921	0.02903	0.03643	0.02900	0.02882	0.04641	0.03991

Table 9: Entropic loss across different models and transaction costs.

The Rigidity of Traditional Models

Traditional models struggle significantly when frictions are introduced:

- **Black-Scholes:** The standard model is entirely blind to trading costs. Its turnover remains rigidly static (approximately 2.0) regardless of the fee magnitude. Because it cannot stop trading, its Entropic Loss severely deteriorates at 50 bps.
- **Leland:** Although theoretically designed to mitigate transaction costs by artificially inflating volatility, the Leland adjustment shows a disappointingly marginal reduction in turnover (dropping only from 2.00 to 1.91 at 50 bps). Consequently, its Entropic Loss remains almost indistinguishable from the standard Black-Scholes.
- **Naiv:** The Naive hedger over-trades massively, maintaining a fixed turnover of over 3.0, making it unusable in high-fee scenarios. Conversely

fee_bps	B-S	GRU	LSTM	Leland	Logistic	MLP	Naive	Unhedged
0.0	1.99999	1.18663	2.62736	1.99999	0.63842	1.55976	3.05360	0.0
5.0	1.99999	1.03586	1.66683	1.98995	0.61721	1.41375	3.05360	0.0
10.0	1.99999	1.03213	1.35474	1.98020	0.77659	1.09848	3.05360	0.0
50.0	1.99999	0.66459	0.65703	1.91080	0.52892	0.55331	3.05360	0.0

Table 10: Turnover across different models and transaction costs.

13.2 Risk aversion and the risk–return trade-off

risk_aversion	B-S	GRU	LSTM	Leland	Logistic	MLP	Naive	Unhedged
1	0.02803	0.02081	0.02078	0.02801	0.02122	0.02077	0.02855	0.03079
5	0.02831	0.02401	0.02629	0.02830	0.02434	0.02388	0.02989	0.03431
10	0.02868	0.02580	0.02636	0.02867	0.02654	0.02575	0.03176	0.03991
25	0.02993	0.02951	0.02947	0.02991	0.03107	0.02829	0.03936	0.07411
50	0.03256	0.03558	0.03551	0.03249	0.04595	0.03161	0.06144	0.16961

Table 11: Entropic loss across different models and risk aversion levels.

Sensitivity to Risk Aversion

The table presents a sensitivity analysis of the Entropic Loss with respect to the risk aversion parameter (λ), ranging from 1 to 50. As expected, increasing the risk aversion heavily penalizes unhedged or poorly hedged positions. The Unhedged portfolio’s loss explodes from 0.03079 to 0.16961, illustrating the severe mathematical penalty applied to unmitigated variance in the final payoff

Low to Moderate Risk Aversion ($\lambda = 1$ to 10)

In regimes of low risk aversion, the deep learning models (MLP, GRU, LSTM) and the Logistic regression significantly outperform the traditional Black-Scholes and Leland baselines. When the penalty for variance is relatively low, the neural networks excel at finding cost-efficient trading trajectories, deliberately accepting a slightly higher hedging error to drastically reduce transaction costs. At $\lambda = 1$, the MLP, GRU, and LSTM achieve an Entropic Loss of approximately 0.020, compared to 0.028 for Black-Scholes.

High Risk Aversion ($\lambda = 25$ to 50)

A structural shift occurs as the risk aversion parameter increases. When the loss function becomes extremely sensitive to replication errors, the cost-saving, sparse-trading strategies of the recurrent networks and the Logistic regression become sub-optimal. At $\lambda = 50$, the Logistic model collapses (0.04595), and both the GRU and LSTM are surpassed by the traditional models. In this extreme regime, the continuous and aggressive rebalancing of Black-Scholes and Leland becomes competitive again, as minimizing the hedging error at all costs outweighs the burden of transaction fees.

The Robustness of the MLP

Notably, the Multi-Layer Perceptron (MLP) remains the most robust and adaptive architecture across the entire spectrum. Even under extreme risk aversion ($\lambda = 50$), it successfully adjusts its strategy to achieve the lowest overall Entropic Loss (0.03161), slightly outperforming both traditional mathematical models (0.03256) and the recurrent architectures.

In the following figure we can see how the BS methods increase their performances by increasing the risk aversion parameter.

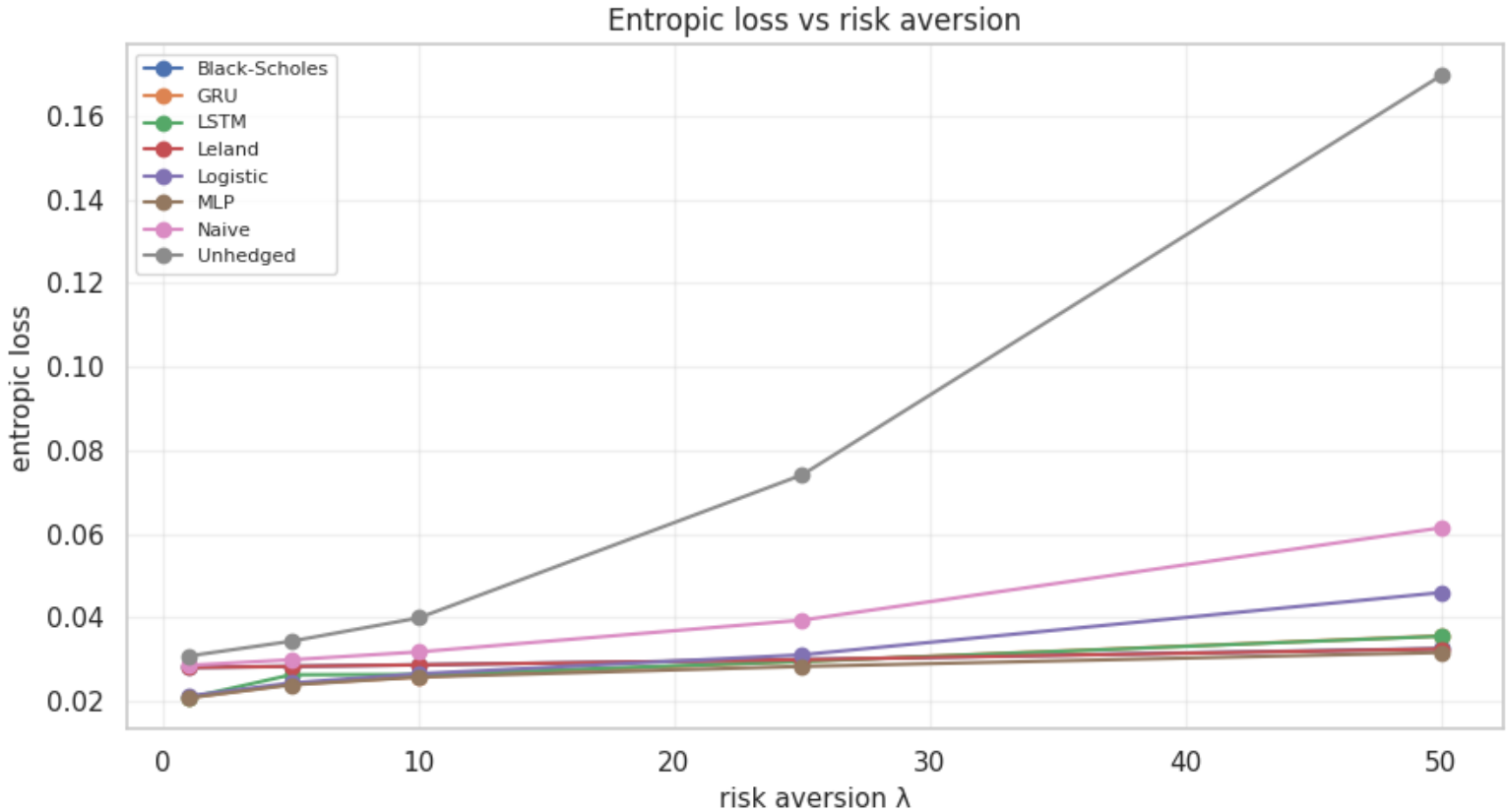


Figure 15: PnL distributions

13.3 Sensitivity to maturity

maturity	B-S	GRU	LSTM	Leland	Logistic	MLP	Naive	Unhedged
10	0.01661	0.01578	0.01592	0.01660	0.01578	0.01577	0.01799	0.01914
20	0.02341	0.02173	0.02204	0.02340	0.02187	0.02144	0.02559	0.03023
30	0.02868	0.02618	0.02577	0.02867	0.02648	0.02560	0.03176	0.03991
60	0.04036	0.03601	0.03652	0.04032	0.03513	0.03549	0.04632	0.06429

Table 12: Entropic loss across different models and option maturities (in days).

Table 12 illustrates the sensitivity of the Entropic Loss to the option's time to maturity, evaluated across horizons of 10, 20, 30, and 60 days.

As theoretically expected, an extension of the time to maturity results in a monotonic increase in the Entropic Loss for all evaluated models. A longer hedging horizon exposes

the portfolio to prolonged stochastic volatility and a higher accumulation of transaction costs, which naturally degrades the overall replication accuracy and increases the final risk penalty. This deterioration is most evident in the Unhedged portfolio, whose loss skyrockets non-linearly from 0.01914 at 10 days to 0.06429 at 60 days, emphasizing the critical necessity of active risk management over extended periods.

Comparing the active strategies, the deep learning architectures (MLP, GRU, LSTM) consistently outperform the traditional mathematical frameworks (Black-Scholes and Leland) across all time horizons. For instance, at the longest maturity (60 days), the neural networks manage to contain the Entropic Loss between 0.035 and 0.036, whereas the standard Black-Scholes model degrades to over 0.040. The Leland adjustment, once again, fails to provide any meaningful advantage over the standard Black-Scholes, yielding nearly identical loss values regardless of the maturity length.

Interestingly, the Logistic regression exhibits remarkably strong performance across all maturities, achieving loss metrics that are highly competitive with, and occasionally marginally better than, the more complex neural architectures (e.g., 0.03513 at 60 days). This suggests that, in this specific simulated environment, a simple linear mapping equipped with an activation function is already capable of capturing the bulk of the cost-saving heuristics required to manage the portfolio over extended horizons.

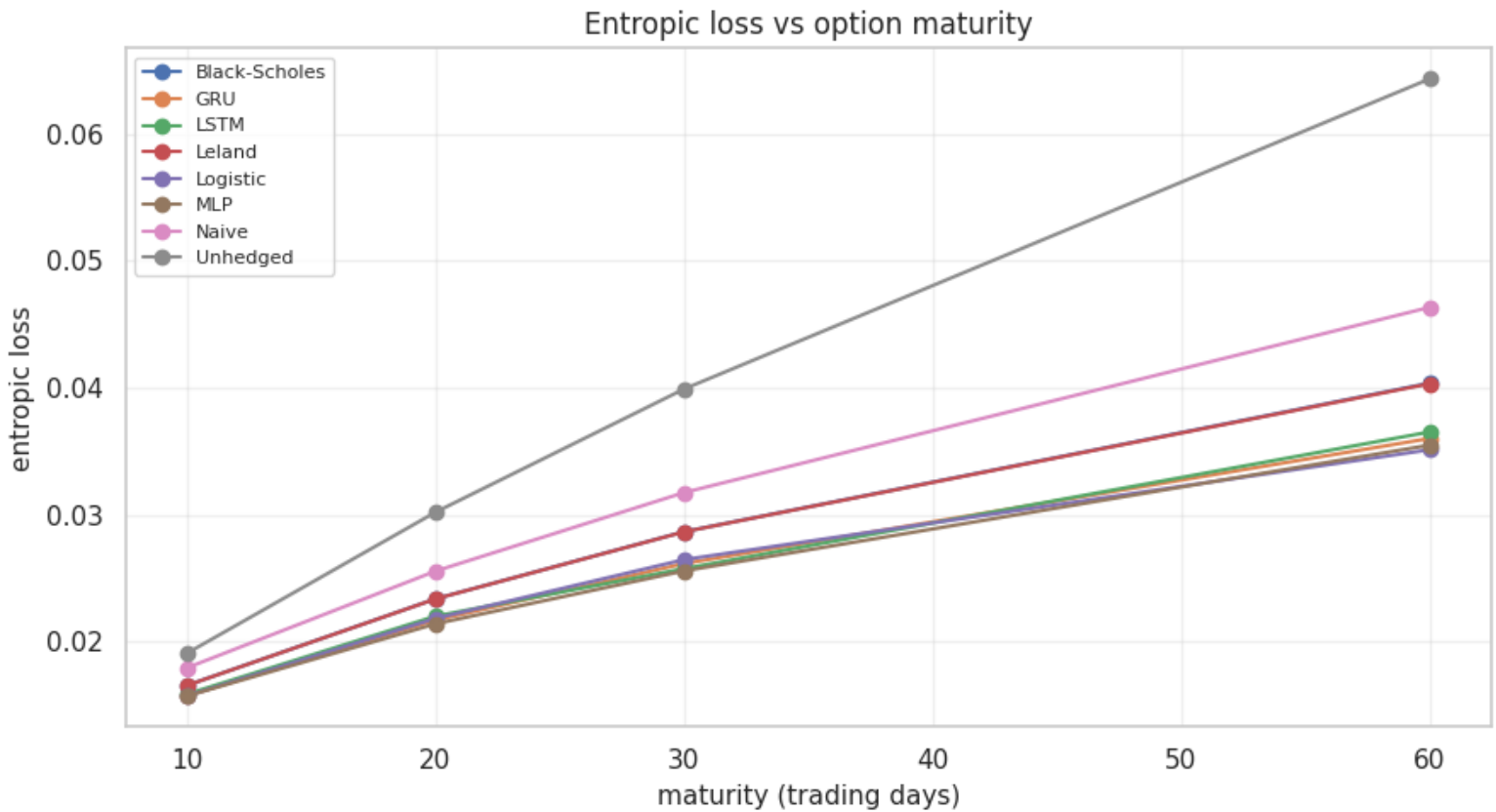


Figure 16: maturity

14 Conclusions

This study presented a comprehensive evaluation of deep learning architectures (MLP, GRU, and LSTM) against traditional mathematical frameworks (Black-Scholes and Le-

land) and baseline strategies (Logistic regression, Naive, and Unhedged) for dynamic option hedging. The primary objective was to assess how these models navigate the fundamental trade-off between minimizing replication error and mitigating transaction costs under varying market conditions.

Overall, the empirical results robustly demonstrate the superiority of neural network models in minimizing the Entropic Loss. In the standard testing environment, the deep learning models consistently outperformed Black-Scholes and Leland. A rigorous bootstrapping analysis confirmed the statistical significance of this outperformance, highlighting the neural networks' ability to learn cost-efficient trading trajectories. Among the evaluated architectures, the Multi-Layer Perceptron (MLP) emerged as the most effective model, achieving the lowest overall Entropic Loss by dynamically and smoothly balancing market responsiveness with transaction cost penalties.

A core advantage of the deep hedging models lies in their behavioral adaptability. While traditional models like Black-Scholes remain structurally rigid—trading continuously and aggressively regardless of market frictions—the neural networks actively adjust their turnover. As demonstrated in the sensitivity analysis, when transaction costs increase, the MLP, GRU, and LSTM autonomously reduce their trading frequency to prevent commission erosion. Conversely, traditional models suffer severe performance degradation in high-fee environments due to their inability to halt trading, and the Leland adjustment fails to provide a meaningful improvement over the standard Black-Scholes baseline.

Furthermore, the sensitivity analysis concerning risk aversion and time to maturity highlighted the structural robustness of the learned models. In regimes of low to moderate risk aversion, all neural architectures and the Logistic regression significantly outperformed traditional baselines by prioritizing cost savings over perfect replication. Under extreme risk aversion, where minimizing hedging error outweighs transaction costs, the recurrent structures (GRU and LSTM) struggled, allowing continuous-time models to become competitive again. Remarkably, the MLP remained the most robust architecture across the entire risk aversion spectrum, successfully adapting its strategy to maintain the lowest Entropic Loss. Finally, testing across different maturities reaffirmed the consistent outperformance of the deep models over traditional frameworks, while also revealing the Logistic regression as a surprisingly strong, albeit rigid, linear baseline for extended horizons.

In conclusion, the application of deep learning to dynamic hedging offers a substantial improvement over traditional frictionless models. By successfully internalizing market frictions and risk preferences directly into the loss function, architectures like the MLP provide a highly adaptable, robust, and cost-efficient solution for real-world option hedging.

References

- [1] Black, F., & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- [2] Merton, R. C. (1973). Theory of rational option pricing. *The Bell Journal of Economics and Management Science*, 4(1), 141–183.
- [3] Buehler, H., Gonon, L., Teichmann, J., & Wood, B. (2019). Deep hedging. *Quantitative Finance*, 19(8), 1271–1291.

- [4] Mazars. (2023). *Deep Learning for Hedging: A Practitioner's Perspective*. Mazars Financial Services.
- [5] Hayne E. Leland (1985) *Option Pricing and Replication with Transaction Costs* The Journal of Finance